

Windows 10/11 SSH Graceful Shutdown and Key-Based Authentication Setup Guide

1. Introduction & Overview

This guide provides step-by-step instructions for configuring a Windows 10 or Windows 11 system to accept remote shutdown commands securely over OpenSSH using key-based authentication.

2. Step 1: Install and Enable OpenSSH Server

To enable SSH capabilities on Windows, install the OpenSSH Server feature and configure the service to run automatically on startup.

Open PowerShell as an Administrator and execute:

```
# Install OpenSSH Server
```

```
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
```

```
# Start SSH service and set startup type to Automatic
```

```
Start-Service sshd
```

```
Set-Service -Name sshd -StartupType 'Automatic'
```

```
# Confirm Firewall Rule is enabled
```

```
Get-NetFirewallRule -Name *ssh*
```

3. Step 2: Configure SSH Key-Based Authentication

Locate the public key (e.g., id_rsa.pub or id_ed25519.pub) on the client system.

Option A: Standard Non-Administrator Accounts

Append the client's public key to the following file path:

```
C:\Users\<Username>\.ssh\authorized_keys
```

Option B: Administrator Accounts (Recommended for Remote Management)

By default, Windows OpenSSH redirects key checking for Administrator accounts to a global configuration file:

```
C:\ProgramData\ssh\administrators_authorized_keys
```

Run the following commands in PowerShell as Administrator: # Ensure the SSH configuration directory exists

```
New-Item -ItemType Directory -Path "C:\ProgramData\ssh" -Force
```

```
# Append the public key to administrators_authorized_keys
```

```
Add-Content -Path "C:\ProgramData\ssh\administrators_authorized_keys" -Value "ssh-ed25519  
AAAAC3NzaC1lZDI1NTE5AAAAI... user@client"
```

4. Step 3: Fix ACL Permissions on Key File

Windows OpenSSH strictly enforces file Access Control List (ACL) permissions. Loose permissions will cause public key authentication to be ignored or rejected.

Execute the following script in PowerShell as Administrator: \$acl = Get-Acl
"C:\ProgramData\ssh\administrators_authorized_keys"

```
$acl.SetAccessRuleProtection($true, $false)
```

```
# Grant access only to SYSTEM and Administrators group
```

```
$administratorsRule = New-Object  
System.Security.AccessControl.FileSystemAccessRule("Administrators", "FullControl", "Allow")
```

```
$systemRule = New-Object System.Security.AccessControl.FileSystemAccessRule("NT  
AUTHORITY\SYSTEM", "FullControl", "Allow")
```

```
$acl.ResetAccessRule($administratorsRule)
```

```
$acl.AddAccessRule($systemRule)
```

```
Set-Acl "C:\ProgramData\ssh\administrators_authorized_keys" $acl
```

5. Step 4: Configure User Rights Assignment (Standard Accounts)

Administrator accounts possess shutdown privileges by default. If using a standard user account, explicit shutdown privileges must be granted:

- 1. Press **Win + R**, type `secpol.msc`, and press Enter.
- 2. Navigate to **Local Policies** → **User Rights Assignment**.
- 3. Open **Shut down the system**.
- 4. Add the targeted standard user account or group.

6. Step 5: Command Reference & Remote Execution

From a remote client (Linux, macOS, or Windows), execute the following command:`ssh -i /path/to/private_key user@windows-ip "shutdown /s /t 0 /f"`

Parameter Reference

Parameter	Description
<code>/s</code>	Initiates a full system shutdown.
<code>/r</code>	Reboots the system instead of shutting down.
<code>/t <seconds></code>	Specifies time delay before shutdown (e.g., <code>/t 0</code> for immediate execution).
<code>/f</code>	Forces running applications to close without prompting users.